The second layer attention counts the bigrams of past tokens, as c approaches infinity, at index i, j the attention is (a constant times) a count of how many positions $p < i$, $\boldsymbol{x}_p = j$ and $\boldsymbol{x}_{p-1} = \boldsymbol{x}_i$. Letting $P = \begin{pmatrix} \mathbf{0} & \mathbf{0} & I^{k\times k} \end{pmatrix}$, the output approaches the probabilities given by empirical bigram statistics.[1] $\square$

Simplified Transformer Architecture. As we see from the construction, there are two main ingredients in the solution realized by the transformer; (1st layer) the ability to look one token back and (2nd layer) the ability to attend to itself. For this reason, we define a *minimal model* that is expressive enough to be able to represent such a solution, but also simple enough to be amenable to analysis. Let e_{x_p} denote the one-hot embedding that corresponds to the state at position $p \in [t]$, and let E be the $\mathbb{R}^{t\times k}$ one-hot embedding matrix. Then the model is defined as:

$$f(E) = \text{mask}\left(EW_k(ME)^T\right)E, \text{ where } M = \begin{pmatrix} v_1 & 0 & \dots & 0 \\ v_2 & v_1 & \dots & 0 \\ \vdots & \vdots & \dots & \vdots \\ v_t & v_{t-1} & \dots & v_1 \end{pmatrix} \in \mathbb{R}^{t\times t} \text{ and } W_k \in \mathbb{R}^{k\times k}. \tag{8}$$

Here mask $(\cdot)$ is a causal mask. Notice that the role of W_k is to mimic the attention mechanism of the second layer and the role of v is that of the positional embeddings.

Fact 2.3. Both the bigrams strategy and the unigrams strategy can be expressed by the minimal model with a simple choice of weights.

- *Bigrams:* For $v = (0, 1, 0 \dots, 0)^\top$, $W_k = I_{k\times k}$, we have $f(E)_{p,i} = \sum_{t'=2}^{p} \mathbb{1}\{x_{t'} = i\}\,\mathbb{1}\{x_{t'-1} = x_p\}$.
- *Unigrams:* For $v = (1, 0, 0 \dots, 0)^\top$, $W_k = 11^T$, we have $f(E)_{p,i} = \sum_{t'=1}^{p} \mathbb{1}\{x_{t'} = i\}$.

3 Empirical Findings and Theoretical Validation

In this section, we present our empirical findings on how transformers succeed in in-context learning Markov Chains, we demonstrate the different learning stages during training and the sudden transitions between them, and draw analytical and empirical insights from the minimal model.

3.1 Transformers in-context learn Markov Chains hierarchically

As can be seen in Figure 3, all the models converge near the Bayesian optimal solution, suggesting that they learn to implement the bigram strategy. Curiously, however, the learning seems to be happening in stages; there is an initial rapid drop and the model quickly finds a better than random solution. Afterwards, there is a long period of only slight improvement before a second rapid drop brings the model close to the Bayes optimal loss. We observe that training a 1-layer transformer fails to undergo a phase transition or converge to the right solution - see Figure 13.

Interestingly, as can be seen from the horizontal lines in Figure 3, the intermediate plateau corresponds to a phase when the model reaches the unigram baseline. We provide evidence that this is not a coincidence, and that after the initial drop in loss, the model's strategy is very similar to the unigram strategy, before eventually being overtaken by the bigram strategy. Some of the strongest such evidence is on the right in Figure 3, where we plot the KL divergence between model's prediction and the two different strategies. For both the strategies, their KL divergence from the model quickly goes down, with the unigram solution being significantly lower. Around the point of the second loss drop, the KL divergence between the model and the bigram solution decreases, while the other one increases, making it clear that the model transitions from the one solution to the other. This final drop is what has been associated to prior work with *induction heads*

[1] Technically, the output of this is probabilities, not log probabilities as generally cross-entropy loss assumes. This can be approximated linearly by setting $P = \begin{pmatrix} b\mathbf{1}^\top\mathbf{1} & \mathbf{0} & aI^{k\times k} \end{pmatrix}$ to change the output from x to $ax+b$. In practice, this approximation can achieve close to Bayes optimal loss.

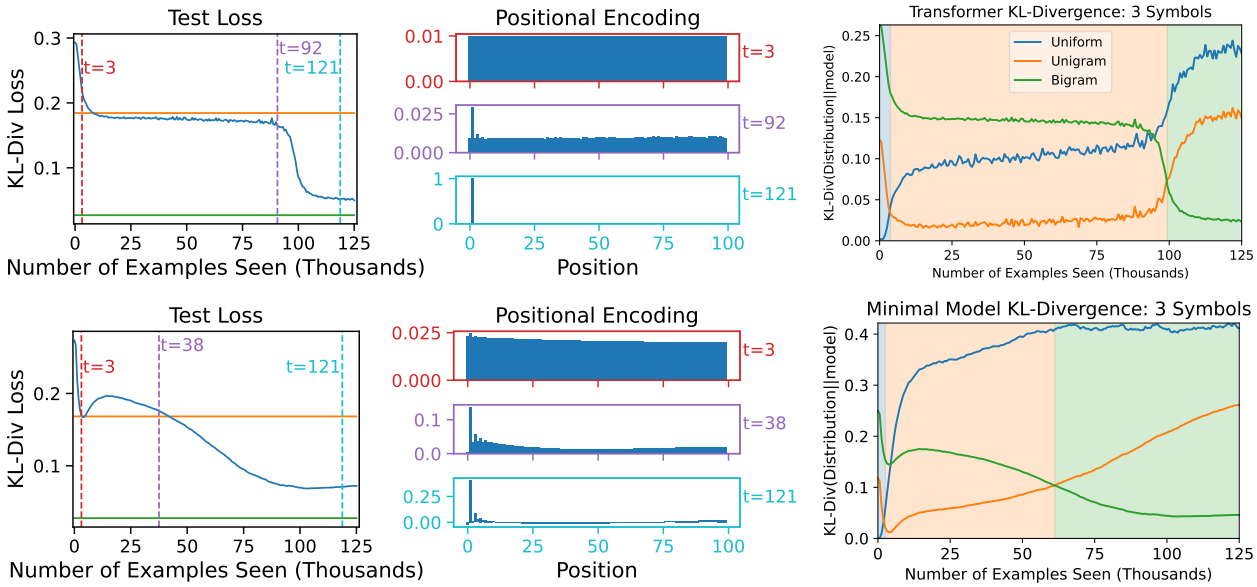

Figure 3: A two layer transformer (**top**) and a minimal model (**bottom**) trained on our in-context Markov Chain task. A comparison of the two layer attention-only transformer and minimal model (8) (with v having constant uniform initialization, and W_K initialized to 0). The graphs on the left are test loss measured by KL-Divergence from the underlying truth. The green line shows the loss of the unigram strategy, and the orange line shows the loss of the bigram strategy. The middle graph shows the effective positional encoding (for the transformer, these are for the first layer, and averaged over all tokens). The graph on the right shows the KL-divergence between the outputs of the models and three strategy. The lower the KL-divergence, the more similar the model is to that strategy.

formation (Olsson et al., 2022); special dedicated heads inside a transformer are suddenly being formed to facilitate in-context learning.

Mechanistic Evidence For Solutions Found By Transformer. To confirm how the two layer attention-only transformer solves ICL-MC, we inspected the attention in each layer throughout training. Figure 2 shows the attention for a particular input during different parts of training. By the end of training, the attention consistently matches that of our construction, with the first layer attending to tokens one in the past, and the second layer attending to tokens that follow the same token as the current one. Note that even if the second layer of the transformer is mostly the same as at the end of training, if the first layer is different, then the weights shown for the second layer attention could differ dramatically. See also Figure 11 in the Appendix that displays how the models perform on different parts of the distribution during training.

Varying the data distribution - Unigrams slow down learning There are several interesting phenomena in the learning scenario that we just described, but it is the second drop (and the preceding plateau) that warrants the most investigation. In particular, one can ask the question: is the unigram solution helpful for the eventual convergence of the model, or is it perhaps just a by-product of the learning procedure?

To answer these questions, we define distributions over Markov chains that are in between the distribution where unigrams is Bayes optimal, and the distribution where unigrams is as good as uniform. As we see in Figure 4a, the transformers that are being trained on the distribution where there is no unigrams “signal" train much faster.

3.2 Theoretical Insights from the Minimal Model

To abstract away some of the many complicated components from the transformer architecture, we focus our attention now to the minimal model of Section 2.3. We train minimal models of eq. (8), starting from a deterministic constant initialization, by minimizing the cross entropy loss with sgd. Full experimental details can be found in the Appendix. Figure 3 (bottom) displays the training curves for the minimal model. Similar to the transformer, the model learns to converge to the bigrams solution, spending however significantly less time, if any, to the unigram solution - even though they can represent it.

We now provide theoretical insights on how training progresses stage by stage and how this is achieved by the synergy between the two layers. As it turns out, there need to be at least two steps of gradient descent in order for both elements of the solution to be formed. The following lemma quantifies this.

Lemma 3.1. *Let the model defined as in eq.* (A.10) *and initialized with* $W_k = c11^T, v = c1^T$. *Then, the after one step of stochastic gradient descent on the margin loss of eq.* (7) *we have:*

$$W_k^{(1)} = \begin{pmatrix} c & c \\ c & c \end{pmatrix} + c\eta \left[O(t^2) \begin{pmatrix} B & A \\ A & B \end{pmatrix} + O(t) \right] \qquad v_j^{(1)} = c + \frac{c\eta}{t} \left[\frac{(t-j+1)(t-j+2)}{2} D + O(t) \right], j \in [t]$$

where $A, B, D > 0$ *with* $B \approx 4A$ *(diagonal bias) and* η *is the learning rate. After the second step,* $v_2^{(2)}$ *becomes dominant, i.e.* $v_2^{(2)} > v_j^{(2)}, j = 1, 3, 4, \ldots, t$.

See Appendix A for the proof. We see that the gradient of W_k at initialization has a clear diagonal bias, and so it starts driving the 2nd layer towards the correct solution - see Constructions in subsection 2.3. The gradient of the positional embeddings, v, however, is rather "uninformative" to the correct coordinate that contains the one back position. Instead, it mostly favors the first position (look at current token), and has a quadratic decay. It is only after the second step that the 1st layer also starts realizing the solution, by growing $v_2^{(2)}$ - see Figure 4b.

There are several interesting observations that follow from Lemma 3.1 and can help us understand better the two stages of learning - both in the toy model and in the transformer:

2nd layer is learnt first It has been observed before in a similar bigram learning setting with a two-layer transformer that the model might be learning first the 2nd layer (Bietti et al., 2023). We also make similar observations in our experiments with the minimal model and the transformers 2, although the evidence is not so clear in the latter case. For the minimal model, the gradient calculations, clearly suggest that starting from a default initialization, it is only the 2nd layer that quickly "picks up" the right solution.

Passing by unigrams. The gradient of the positional embeddings v at initialization delivers no information about the optimal solution, but has a quadratic structure instead that mostly favors v_1. If the gradient of the 2nd layer is small in scale (which can happen either due to the initialization of v or to a small learning rate), then W_k will not deviate much from the uniform initialization. Notice, then, that a uniform W_k together with a biased first coordinate of v correspond precisely to the unigram solution in this model. Thus, it is clear in this case that W_k not being aligned with the solution yet, not only slows down v, but also biases it towards a simpler solution. We believe that the same mechanism, perhaps manifested differently, is also responsible for the hierarchical learning in the transformer.

Even/odd pattern. An interesting observation that comes out from the calculations is the form of v_j after 2 steps; the gradient amplifies much more the even coordinates than the odd ones, with a scale that follows a geometric series. In fact, the ratio is closely related to the moments of the eigenvalue of the transition matrix. This way the second coordinate starts growing larger than the rest and the model eventually learns to represent the bigrams solution. As a byproduct, however, the rest of the even coordinates also grow in magnitude, despite not being part of the optimal solution. Perhaps surprisingly, we are able to also identify the same spurious pattern in the transformers! In the transformer, if the positional embeddings are